

DevSight — Concept of Operations (ConOps)

1. Project Overview

Project Name

DevSight

Project Type

Full-stack developer operations and security awareness platform.

Vision Statement

DevSight is designed to provide developers and engineering teams with a centralized platform for monitoring projects, tracking operational activity, managing tasks, and integrating DevSecOps-oriented workflows into a modern web application.

The platform combines concepts from:

- project management systems
- developer dashboards
- security tooling
- operational monitoring
- CI/CD visibility

into a unified engineering-focused SaaS platform.

2. Primary Objectives

Core Goals

1. Provide a centralized engineering dashboard

Users should be able to view project activity, tasks, security findings, and operational data from a single interface.

2. Simulate real-world engineering workflows

The application should reflect modern software engineering practices including:

- authentication
- CI/CD integration
- security scanning
- API-driven architecture
- database persistence
- infrastructure awareness

3. Demonstrate full-stack engineering competency

DevSight should serve as a portfolio-quality demonstration of:

- frontend engineering
- backend architecture
- DevOps concepts
- security awareness
- system design
- operational tooling

4. Maintain extensibility

The system architecture should allow future expansion without major rewrites.

3. Target Users

Primary Users

Developers

Users managing projects, tasks, deployments, and engineering workflows.

Students & Learners

Users learning:

- DevOps
- software engineering
- full-stack development
- security concepts

Engineering Teams (Future Goal)

Potential collaborative workflow support.

4. System Scope

Included in Current Scope

Authentication

- User registration
- User login
- JWT authentication
- Protected routes

Project Management

- Project creation
- Project tracking
- Task management
- Status tracking

Dashboard System

- User dashboard
- Project overview
- Activity feed
- Security panel

DevSecOps Features

- Static analysis visibility
- Security scan integration
- Pipeline status awareness
- Vulnerability reporting concepts

Infrastructure

- PostgreSQL database
 - Dockerized services
 - CI/CD workflows
 - environment variable management
-

5. Baseline Functional Requirements

Authentication Requirements

FR-1

Users shall be able to register an account.

FR-2

Users shall be able to login using email and password.

FR-3

Passwords shall be securely hashed before storage.

FR-4

Authenticated users shall receive JWT access tokens.

FR-5

Protected pages shall require authentication.

Dashboard Requirements

FR-6

Users shall be able to view a personalized dashboard.

FR-7

The dashboard shall display:

- active projects
 - open tasks
 - recent activity
 - security information
-

Project Management Requirements

FR-8

Users shall be able to create projects.

FR-9

Users shall be able to edit project details.

FR-10

Users shall be able to create tasks associated with projects.

FR-11

Users shall be able to update task status.

API Requirements

FR-12

Backend services shall expose RESTful API endpoints.

FR-13

Frontend components shall communicate with backend APIs using HTTP requests.

FR-14

API routes shall validate incoming request data.

Security Requirements

FR-15

The application shall use environment variables for sensitive configuration.

FR-16

Static analysis tooling shall be integrated into development workflows.

FR-17

Authentication endpoints shall reject invalid credentials.

6. Non-Functional Requirements

Security

NFR-1

Sensitive credentials shall not be stored in source control.

NFR-2

Passwords shall never be stored in plaintext.

NFR-3

Protected routes shall enforce authentication checks.

Maintainability

NFR-4

The backend architecture shall remain modular.

NFR-5

The frontend architecture shall use reusable React components.

NFR-6

The project shall maintain readable documentation.

Scalability

NFR-7

The application architecture shall support future feature expansion.

NFR-8

Database models shall support relational growth.

Reliability

NFR-9

CI/CD pipelines shall validate frontend and backend builds.

NFR-10

The application shall include health monitoring endpoints.

Usability

NFR-11

The UI shall remain clean and intuitive.

NFR-12

The application shall initially prioritize readability over visual complexity.

7. Technical Stack

Frontend

- React
- TypeScript
- Vite
- TailwindCSS v4

Backend

- FastAPI
- SQLAlchemy
- Uvicorn

Database

- PostgreSQL

Infrastructure

- Docker
- GitHub Actions

DevSecOps Tooling

- Ruff
 - Bandit
 - SonarLint
-

8. Development Phases

Phase 1 — Infrastructure Foundation

Completed:

- repository setup
 - frontend setup
 - backend setup
 - database integration
 - Docker setup
 - CI/CD setup
 - DevSecOps tooling integration
-

Phase 2 — Authentication & Core SaaS Functionality

Planned:

- backend architecture cleanup
 - user model
 - password hashing
 - JWT authentication
 - login/register endpoints
 - frontend auth pages
 - protected routes
-

Phase 3 — Core Dashboard Features

Planned:

- project management
 - task systems
 - dashboard analytics
 - activity feeds
-

Phase 4 — Advanced DevOps & Security Features

Planned:

- security dashboards
 - deployment monitoring
 - vulnerability visualization
 - advanced analytics
 - theme system
-

9. UI/UX Direction

Development Theme

Light Minimal

Future Stretch Goal

Theme system supporting:

- Light Minimal
 - Modern Dark
 - Glassmorphism
-

10. Long-Term Vision

DevSight aims to evolve into a portfolio-quality engineering platform demonstrating:

- full-stack engineering
- DevSecOps practices
- operational awareness

- scalable architecture
- secure software engineering principles
- modern SaaS development workflows.